

Welcome to CSC331 Fall 2025 Data Structures

Professor Kevin Byron (kbyron@bmcc.cuny.edu)

Department of CIS, Room F-1030-I

Office hours: Mon 2pm-5pm (in-person and Zoom)

Office phone: 212-220-1490

Borough of Manhattan Community College/CUNY

CSC331 Fall 2025

Data Structures

Week 1:

Review course syllabus
Review BMCC semester calendar
Brightspace LMS
C++ program execution
First program assignment
Software development overview/review

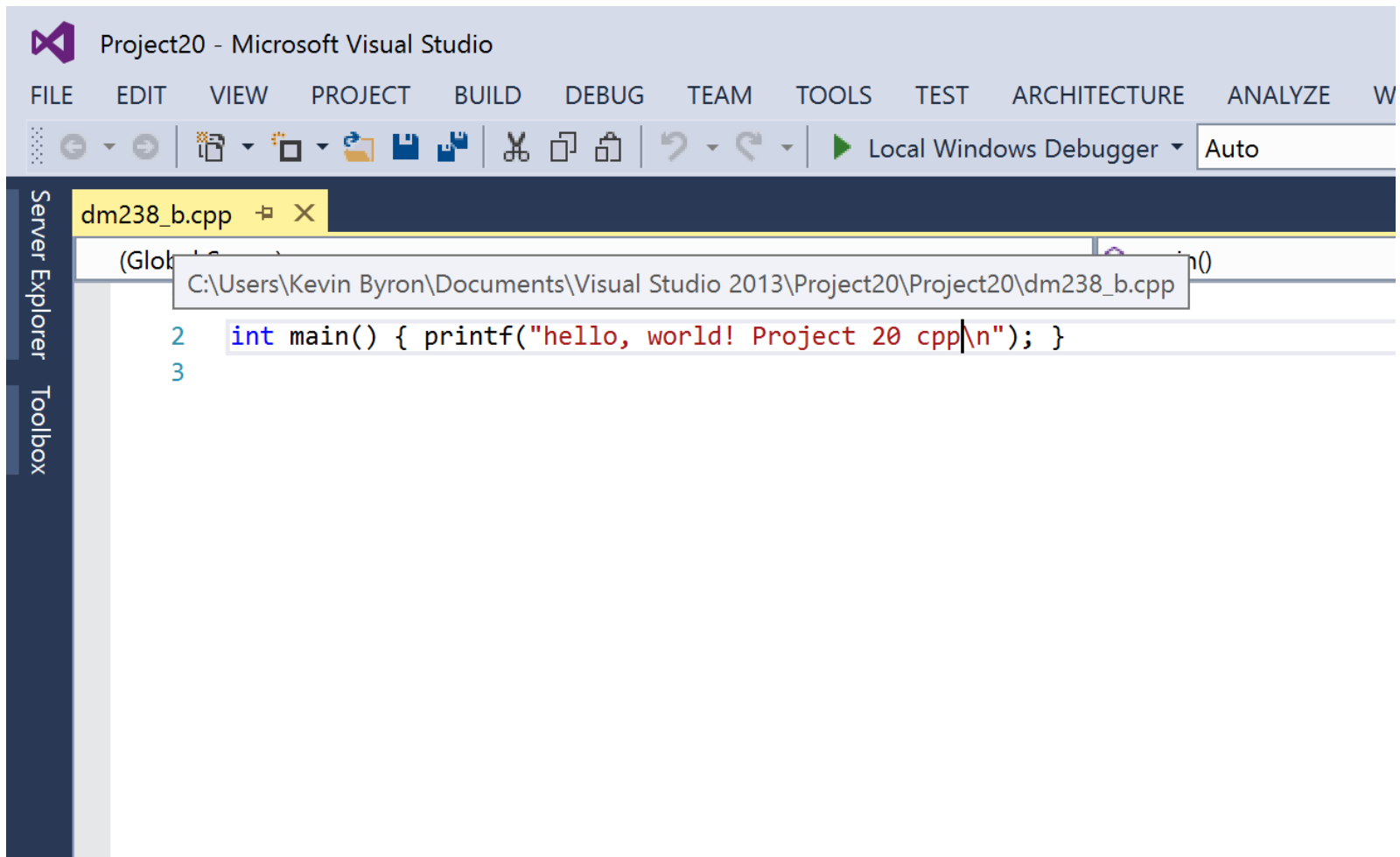
Professor Kevin Byron
Department of CIS
Borough of Manhattan Community College/CUNY

Reminders

- No class Mon 9-1-2025, Mon 9-22-2025 thru Wed 9-24-2025
- Program 1 due date
 - 1159pm Fri 9-19-2025
 - late submission receives no credit
- Exam 1
 - Section 501 Tue 9-30-2025 – Zoom 630pm-730pm
 - Section 172 Wed 10-8-2025 – In person 630pm-730pm
 - Closed book, closed notes, no devices
 - Section 500 Wed 10-8-2025 – In person 1010am-1110am
 - Closed book, closed notes, no devices
- Posted on Brightspace
 - Faculty information, Course syllabus, program 1 specifications, Malik textbook 2nd edition, Week 1 lecture slides, sample c++ hello program (dm468.cpp), personType.h header file, sample Prog1 screen capture report

Locate and run executable code

1. “hover” over source file name to reveal file location



Locate and run executable code

2. In CMD, go to VS project directory & find executable
3. Go to that directory and run the executable

 Command Prompt

```
Microsoft Windows [Version 10.0.18362.592]  
(c) 2019 Microsoft Corporation. All rights reserved.
```

```
C:\Users\Kevin Byron>cd "Documents\Visual Studio 2013\Project20"
```

```
C:\Users\Kevin Byron\Documents\Visual Studio 2013\Project20>dir /S *exe  
Volume in drive C is Windows  
Volume Serial Number is 2054-9A54
```

```
Directory of C:\Users\Kevin Byron\Documents\Visual Studio 2013\Project20\Debug
```

```
02/01/2020  03:32 PM                61,440 Project20.exe  
                1 File(s)                61,440 bytes
```

```
Total Files Listed:
```

```
        1 File(s)                61,440 bytes  
        0 Dir(s)  364,525,203,456 bytes free
```

```
C:\Users\Kevin Byron\Documents\Visual Studio 2013\Project20>cd Debug
```

```
C:\Users\Kevin Byron\Documents\Visual Studio 2013\Project20\Debug>Project20 > output.txt
```

```
C:\Users\Kevin Byron\Documents\Visual Studio 2013\Project20\Debug>type output.txt  
hello, World! Project 20 cpp
```

```
C:\Users\Kevin Byron\Documents\Visual Studio 2013\Project20\Debug>
```

Overview/Review

Software Life Cycle

A program goes through many phases from the time it is first conceived until the time it is retired, called the *life cycle* of the program. The three fundamental stages through which a program goes are *development*, *use*, and *maintenance*. Usually a program is initially conceived by a software developer because a customer has some problem that needs to be solved and the customer is willing to pay money to have it solved. The new program is created in the *software development* stage. The next section describes this stage in some detail.

Overview/Review

Software Development Phase

Software engineers typically break the software development process into the following four phases:

- Analysis
- Design
- Implementation
- Testing and debugging

Overview/Review

Analysis

Analyzing the problem is the first and most important step. This step requires you to do the following:

- Thoroughly understand the problem.
- Understand the problem requirements. Requirements can include whether the program requires interaction with the user, whether it manipulates data, whether it produces output, and what the output looks like.

Design

After you carefully analyze the problem, the next step is to design an algorithm to solve the problem. If you broke the problem into subproblems, you need to design an algorithm for each subproblem.

Overview/Review

STRUCTURED DESIGN

Dividing a problem into smaller subproblems is called **structured design**. The structured design approach is also known as **top-down design**, **stepwise refinement**, and **modular programming**. In structured design, the problem is divided into smaller problems. Each subproblem is then analyzed, and a solution is obtained to solve the subproblem. The solutions of all the subproblems are then combined to solve the overall problem. This process of implementing a structured design is called **structured programming**.

OBJECT-ORIENTED DESIGN

In object-oriented design (OOD), the first step in the problem-solving process is to identify the components called objects, which form the basis of the solution, and determine how these objects interact with one another. For example, suppose you want to write a program that automates the video rental process for a local video store. The two main objects in this problem are the video and the customer.

Overview/Review

Implementation

In the *implementation* phase, you write and compile programming code to implement the classes and functions that were discovered in the design phase.

Testing and Debugging

The term *testing* refers to testing the correctness of the program; that is, making sure that the program does what it is supposed to do. The term *debugging* refers to finding and fixing the errors, if they exist.

There are two types of testing—*black-box* testing and *white-box* testing. In black-box testing, you do not know the internal working of the algorithm or function. You know only what the function does. Black-box testing is based on inputs and outputs.

White-box testing relies on the internal structure and implementation of a function or algorithm. The objective is to ensure that every part of the function or algorithm is executed at least once. Suppose that you want to ensure whether an **if** statement works properly. The test cases must consist of at least one input for which the **if** statement evaluates to **true** and at least one case for which it evaluates to **false**. Loops and other structures can be tested similarly.

Overview/Review

Algorithm Analysis: The Big-O Notation

Just as a problem is analyzed before writing the algorithm and the computer program, after an algorithm is designed it should also be analyzed. Usually, there are various ways to design a particular algorithm. Certain algorithms take very little computer time to execute, whereas others take a considerable amount of time.

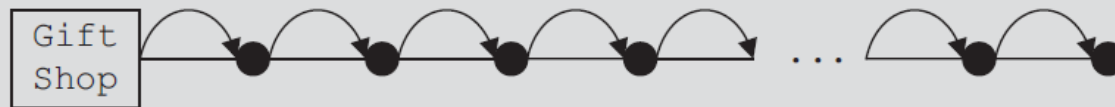


FIGURE 1-2 Package delivering scheme

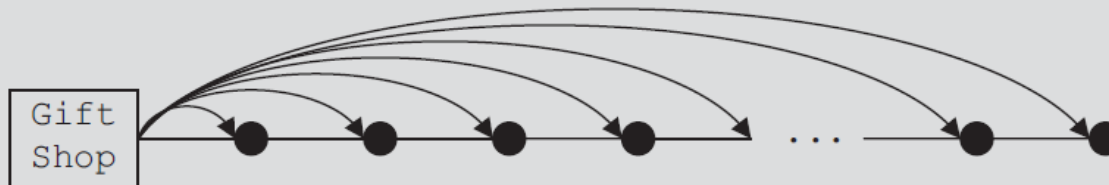


FIGURE 1-3 Another package delivery scheme

Overview/Review

Now suppose that there are n packages to be delivered to n houses, and each house is one mile apart from each other, as shown in Figure 1-1. If the packages are delivered using the first scheme, the following equation gives the total distance traveled:

$$1 + 1 + \dots + 1 + n = 2n \quad (1-1)$$

If the packages are delivered using the second method, the distance traveled is:

$$2 \cdot (1 + 2 + 3 + \dots + n) = 2 \cdot (n(n + 1)/2) = n^2 + n \quad (1-2)$$

TABLE 1-1 Various values of n , $2n$, n^2 , and $n^2 + n$

n	$2n$	n^2	$n^2 + n$
1	2	1	2
10	20	100	110
100	200	10,000	10,100
1000	2000	1,000,000	1,001,000
10,000	20,000	100,000,000	100,010,000

Overview/Review

EXAMPLE 1-1

Consider the following algorithm. (Assume that all variables are properly declared.)

```
cout << "Enter two numbers";           //Line 1

cin >> num1 >> num2;                   //Line 2

if (num1 >= num2)                       //Line 3
    max = num1;                         //Line 4
else                                    //Line 5
    max = num2;                         //Line 6

cout << "The maximum number is: " << max << endl; //Line 7
```

Line 1 has one operation, <<; Line 2 has two operations; Line 3 has one operation, >=; Line 4 has one operation, =; Line 6 has one operation; and Line 7 has three operations. Either Line 4 or Line 6 executes. Therefore, the total number of operations executed in the preceding code is $1 + 2 + 1 + 1 + 3 = 8$. In this algorithm, the number of operations executed is fixed.

Overview/Review

TABLE 1-2 Growth rates of various functions

n	$\log_2 n$	$n \log_2 n$	n^2	2^n
1	0	0	1	2
2	1	2	2	4
4	2	8	16	16
8	3	24	64	256
16	4	64	256	65,536
32	5	160	1024	4,294,967,296

Overview/Review

TABLE 1-3 Time for $f(n)$ instructions on a computer that executes 1 billion instructions per second

n	$f(n) = n$	$f(n) = \log_2 n$	$f(n) = n \log_2 n$	$f(n) = n^2$	$f(n) = 2^n$
10	0.01 μ s	0.003 μ s	0.033 μ s	0.1 μ s	1 μ s
20	0.02 μ s	0.004 μ s	0.086 μ s	0.4 μ s	1ms
30	0.03 μ s	0.005 μ s	0.147 μ s	0.9 μ s	1s
40	0.04 μ s	0.005 μ s	0.213 μ s	1.6 μ s	18.3min
50	0.05 μ s	0.006 μ s	0.282 μ s	2.5 μ s	13 days
100	0.10 μ s	0.007 μ s	0.664 μ s	10 μ s	4×10^{13} years
1000	1.00 μ s	0.010 μ s	9.966 μ s	1ms	
10,000	10 μ s	0.013 μ s	130 μ s	100ms	
100,000	0.10ms	0.017 μ s	1.67ms	10s	
1,000,000	1 ms	0.020 μ s	19.93ms	16.7m	
10,000,000	0.01s	0.023 μ s	0.23s	1.16 days	
100,000,000	0.10s	0.027 μ s	2.66s	115.7 days	

Overview/Review

TABLE 1-4 Growth rate of n^2 and $n^2 + 4n + 20$

n	$g(n) = n^2$	$f(n) = n^2 + 4n + 20$
10	100	160
50	2500	2720
100	10,000	10,420
1000	1,000,000	1,004,020
10,000	100,000,000	100,040,020

Overview/Review

EXAMPLE 1-6

In the following, $f(n)$ is a nonnegative real-valued function.

Function

$f(n) = an + b$, where a and b are real numbers and a is nonzero.

$$f(n) = n^2 + 5n + 1$$

$$f(n) = 4n^6 + 3n^3 + 1$$

$$f(n) = 10n^7 + 23$$

$$f(n) = 6n^{15}$$

Big-O

$$f(n) = O(n)$$

$$f(n) = O(n^2)$$

$$f(n) = O(n^6)$$

$$f(n) = O(n^7)$$

$$f(n) = O(n^{15})$$

Overview/Review

EXAMPLE 1-8

Consider the following code, where `m` and `n` are `int` variables and their values are nonnegative:

```
for (int i = 0; i < m; i++)          //Line 1
    for (int j = 0; j < n; j++)      //Line 2
        cout << i * j << endl;      //Line 3
```

This code contains nested `for` loops. The outer `for` loop, at Line 1, executes m times. For each iteration of the outer loop, the inner loop, at Line 2, executes n times. For each iteration of the inner loop, the output statement in Line 3 executes. It follows that the total number of iterations of the nested `for` loop is mn . So the number of times the statement in Line 3 executes is mn . Therefore, this algorithm is $O(mn)$. Note that if $m = n$, then this algorithm is $O(n^2)$.

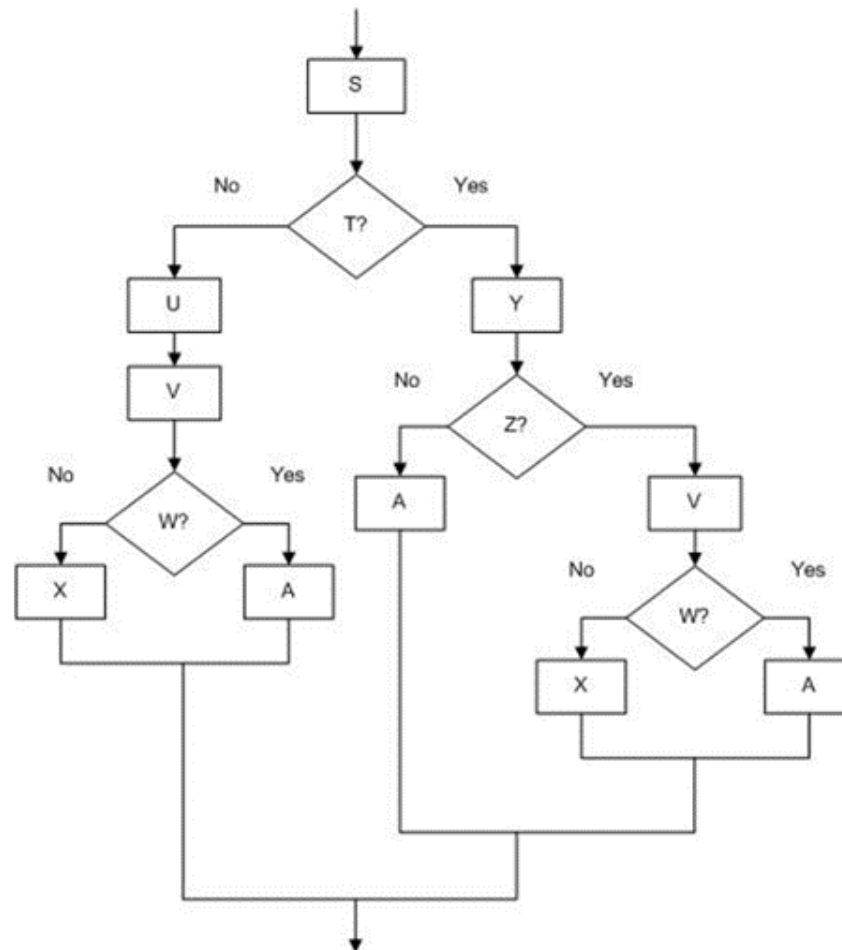
Overview/Review

EXERCISES

1. Mark the following statements as true or false.
 - a. The life cycle of software refers to the phases from the point the software was conceived until it is retired.
 - b. The three fundamental stages of software are development, use, and discard.
 - c. The expression $4n + 2n^2 + 5$ is $O(n)$.
2. What is black-box testing?
3. What is white-box testing?

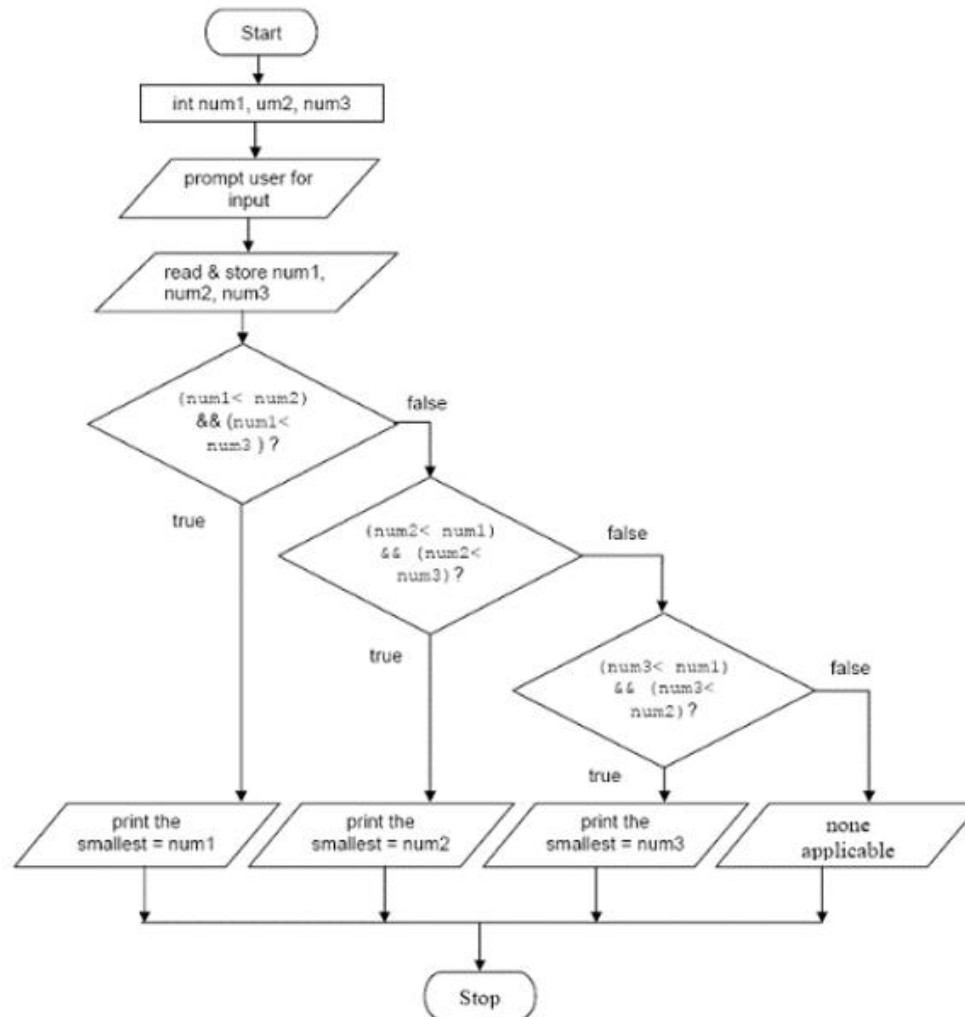
Overview/Review

- How many test cases are needed to test this logic using white-box testing?



Overview/Review

- How many test cases are needed to test this logic using white-box testing?



Overview/Review

5. Each of the following expressions represents the number of operations for certain algorithms. What is the order of each of these expressions?
- a. $n^2 + 6n + 4$
 - b. $5n^3 + 2n + 8$
 - c. $(n^2 + 1)(3n + 5)$
 - d. $5(6n + 4)$
 - e. $n + 2\log_2 n - 6$
 - f. $4n \log_2 n + 3n + 8$

Overview/Review

10. Characterize the following algorithm in terms of Big-O notation. Also find the exact number of additions executed by the loop. (Assume that all variables are properly declared.)

```
for (int i = 1; i <= n; i++)  
    sum = sum + i * (i + 1);
```

11. Characterize the following algorithm in terms of Big-O notation. Also find the exact number of additions, subtractions, and multiplications executed by the loop. (Assume that all variables are properly declared.)

```
for (int i = 5; i <= 2 * n; i++)  
    cout << 2 * n + i - 1 << endl;
```

12. Characterize the following algorithm in terms of Big-O notation.

```
for (int i = 1; i <= 2 * n; i++)  
    for (int j = 1; j <= n; j++)  
        cout << 2 * i + j;  
cout << endl;
```

13. Characterize the following algorithm in terms of Big-O notation.

```
for (int i = 1; i <= n; i++)  
    for (int j = 1; j <= n; j++)  
        for (int k = 1; k <= n; k++)  
            cout << i + j + k;
```

Overview/Review

- Other Chapter 1 practice exercises
 - 14, 15, 16, 19 on Malik pages 53 through 56

Assignment

- Read Malik Chapter 1 pages 2-33